The OpenClaw Blueprint

Build Your AI Bot That Works While You Sleep

by dossdyai

Introduction

What if your AI assistant could send emails, deploy websites, process payments, monitor the internet for you — and do all of it while you sleep?

This guide shows you exactly how to set that up. No coding background required. Every step is explained simply, and by the end you'll have a real AI bot running 24/7 on your own accounts.

This is not theory. Every setup in this guide was built and tested in real time.

Chapter 1: What Is OpenClaw?

OpenClaw is a personal AI gateway. It connects a powerful AI model (Claude by Anthropic) to your real life — your messages, files, calendar, email, and tools.

Think of it like hiring a very capable assistant who:

- Lives inside your Telegram
- Has access to your computer
- Can run code, browse the web, send emails, and deploy apps
- Works 24/7 — even when you're asleep

The key difference from ChatGPT or other AI tools: **OpenClaw acts, not just responds.** It can take real actions in the real world on your behalf.

Chapter 2: Day 0 — Your First Setup

What You Need

- A Mac or Linux computer (or a VPS — covered in Chapter 10)
- A Telegram account
- About 30 minutes

Step 1 — Install OpenClaw

```
npm install -g openclaw
```

Step 2 — Run the Setup Wizard

```
openclaw setup
```

This will ask you to:

- Connect your Telegram bot (create one via @BotFather)
- Set your AI model (Claude Sonnet recommended)

- Choose your workspace directory

Step 3 — Create Your Identity Files

OpenClaw reads files in your workspace to understand who it is and who it's helping.

Create these two files in your workspace:

SOUL.md — the bot's personality

```
# SOUL.md
Be genuinely helpful, not performatively helpful.
Have opinions. Be resourceful before asking.
Earn trust through competence.
```

USER.md — information about you

```
# USER.md
- Name: [Your Name]
- Timezone: [Your timezone]
- Preferences: [How you like to receive information]
```

Step 4 — Start the Gateway

```
openclaw gateway start
```

Now send a message to your Telegram bot. It should respond.

You're live.

Chapter 3: Telegram Setup — Your Command Center

Telegram is how you talk to your bot. But you can go much further than a simple chat.

Direct Chat

The default setup — message your bot directly. Works immediately after setup.

Group with Topics (Recommended)

Create a Telegram group, add your bot, and enable **Topics**. This lets you create separate workspaces:

- 💼 **Business** — product and revenue work
- 🔧 **Dev** — coding and deployment tasks
- 📊 **Research** — monitoring and analysis
- 📋 **General** — everything else

How to set this up:

1. Open Telegram → New Group → add your bot
2. Tap the group name → Edit → enable Topics
3. Create topics for each area

4. Make sure bot privacy is OFF (via @BotFather → your bot → Group Privacy → Off)

Each topic acts as an isolated workspace. Your bot keeps context separate per topic — so business conversations don't mix with dev work.

Chapter 4: Notion Integration

Notion becomes your bot's second brain — a place to store projects, databases, and notes that persist across sessions.

Setup

1. Go to notion.so/my-integrations
2. Create a new integration → copy the API key
3. Run: `openclaw config set NOTION_API_KEY your_key_here`
4. Share any Notion page with your integration to grant access

What Your Bot Can Do

- Create and update pages
- Query databases
- Add entries, check boxes, write notes
- Organize your workspace on command

Recommended Workspace Structure

- **Hub page** — links to all projects
- **Project pages** — one per active project
- **Database** — track tasks, ideas, content

Chapter 5: Memory System — How Your Bot Remembers

By default, AI models forget everything between sessions. OpenClaw solves this with files.

Three Layers of Memory

Layer 1: Daily Notes `memory/YYYY-MM-DD.md` — raw log of what happened each day. Created automatically.

Layer 2: Long-Term Memory `MEMORY.md` — your curated, distilled knowledge. Key decisions, preferences, ongoing context.

Layer 3: Knowledge Graph (PARA System) A folder structure that mirrors how you think:

```
~/life/
  Projects/    ← active, time-bound work
  Areas/       ← ongoing responsibilities
  Resources/   ← reference material
  Archives/    ← completed work
```

Create a folder for every important person, project, or company. Add a `summary.md` inside.

Nightly Consolidation

Set a cron job to run at 2AM:

> *"Review today's conversations. Update ~/life/ with new decisions, context, and lessons."*

Your bot builds its own memory over time — getting smarter about you the longer it runs.

Chapter 6: Behavior Rules — Making Your Bot Reliable

Your bot follows rules defined in `AGENTS.md`. Here's what matters most:

Plan Before Acting

For any multi-step task, write a plan to `tasks/todo.md` first. Review it before starting. This prevents the bot from going in the wrong direction on complex work.

Self-Improvement Loop

After any correction, update `tasks/lessons.md` with the pattern. Over time, your bot learns your preferences and stops making the same mistakes.

Verify Before Done

Never mark a task complete without proving it works. Run checks, review output, demonstrate results.

Autonomous Problem Solving

For bugs and technical problems: just fix them. Don't ask for hand-holding. Point at logs, errors, and outputs — then resolve.

Task Management

1. Write plan → `tasks/todo.md`
2. Confirm with you before starting
3. Mark items complete as you go
4. Update `tasks/lessons.md` after corrections

Chapter 7: Automation — Cron Jobs & Monitoring

Your bot can run tasks on a schedule without you doing anything.

Setting Up a Cron Job

```
openclaw cron add \
  --name "morning-scout" \
  --cron "0 8 * * *" \
  --tz "Asia/Bangkok" \
  --message "Fetch latest AI/tech news from HackerNews and Reddit. Summarize top 5
findings. Send digest to Telegram."
```

Useful Automations to Set Up

Morning News Scout (8AM daily)

> *Scan HackerNews, Reddit, TechCrunch for your chosen topics. Send Telegram digest.*

Nightly Memory Update (2AM daily)

> *Review today's sessions. Update ~/life/ knowledge graph. Distill key learnings.*

Weekly Review (Sunday 9AM)

> *Summarize week's progress. Update project status. Flag anything that needs attention.*

Heartbeat Checks

Edit `HEARTBEAT.md` with periodic tasks. OpenClaw polls this file regularly and acts on anything listed.

Chapter 8: GitHub + Vercel — Your Bot Deploys Code

Give your bot its own GitHub account so it can push code and deploy apps — without touching your personal account.

Step 1 — Create a Bot GitHub Account

- Use a dedicated email (e.g. yourbot@gmail.com)
- Username: yourname-bot
- Generate a Personal Access Token: Settings → Developer Settings → Tokens (classic)
- Scopes: `repo`, `workflow`, `read:org`

Step 2 — Connect Vercel

- Sign up at vercel.com using the bot's GitHub account
- Generate a Vercel token: Account → Settings → Tokens
- Store both tokens in `~/.openclaw/secrets/accounts.md`

Step 3 — Your Bot Can Now

- Create GitHub repositories
- Push code changes
- Deploy websites and apps to Vercel
- Get live URLs automatically

Why Separate Accounts?

If something goes wrong, you revoke the bot's tokens. Your personal GitHub stays untouched. Contained blast radius.

Chapter 9: Stripe — Sell Things While You Sleep

This is where the money comes in.

Step 1 — Create a Stripe Account

- stripe.com → Create account
- Use your bot's email or a business email
- Complete the onboarding (website required — use your GitHub Pages site)

Step 2 — Get Your API Keys

Dashboard → Developers → API Keys

- **Publishable key** — goes in your frontend
- **Secret key** — stored securely, never in code

Store in `~/.openclaw/secrets/accounts.md` (chmod 600).

Step 3 — Create a Product

Dashboard → Product Catalog → Add Product

- Name: your guide/course/service
- Price: set your amount
- Type: One-time payment

Step 4 — Build a Checkout Page

Basic HTML with Stripe.js:

```
<button id="checkout-btn">Buy Now — $39</button>
<script src="https://js.stripe.com/v3/"></script>
<script>
const stripe = Stripe('your_publishable_key');
document.getElementById('checkout-btn').onclick = async () => {
  const res = await fetch('/create-checkout', {method: 'POST'});
  const {url} = await res.json();
  window.location = url;
};
</script>
```

Step 5 — Auto-Deliver Your Product

Set up a webhook: when payment succeeds → send product (PDF download link, course access, etc.) via email.

Test Before Going Live

Use Stripe's test card: `4242 4242 4242 4242`, any future date, any CVC. Run a full test purchase before accepting real payments.

Chapter 10: VPS Hosting — Run 24/7 for $5/Month

GitHub Pages and Vercel are great for websites. But to run OpenClaw 24/7 as a server, you need a VPS.

Recommended Providers

- **Hetzner** — cheapest, European servers, €4/month
- **DigitalOcean** — easy to use, $6/month
- **Vultr** — flexible, $5/month

Setup Steps

1. Create a VPS Choose Ubuntu 22.04. Pick the cheapest tier (1 CPU, 1GB RAM is enough).

2. Connect via SSH

```
ssh root@your-server-ip
```

3. Install Dependencies

```
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs git
npm install -g openclaw
```

4. Transfer Your Workspace

```
scp -r ~/.openclaw root@your-server-ip:~/
```

5. Run as a Service

```
# Install PM2 to keep it running
npm install -g pm2
pm2 start "openclaw gateway start" --name openclaw
pm2 save
pm2 startup
```

6. Done Your bot now runs 24/7, even when your laptop is closed.

Cost: ~$5/month. Revenue potential: unlimited.

Chapter 11: Your First Product

You now have everything:

- ✅ AI bot running 24/7
- ✅ GitHub for code
- ✅ Vercel for deployment
- ✅ Stripe for payments
- ✅ Memory system building over time

Build your first product in this order:

1. **Decide what to sell** — guide, course, tool, service
2. **Create the product** — write it, build it (your bot helps)
3. **Create Stripe product** — set price, get payment link
4. **Build landing page** — simple, clear, one call to action
5. **Deploy** — GitHub push → Vercel live in 60 seconds
6. **Set up delivery** — webhook → auto-send on payment
7. **Test** — buy your own product with test card
8. **Launch** — share the link

The entire system runs on its own after launch. Your bot handles support questions. Stripe handles payments. The webhook handles delivery.

You handle nothing — except cashing out.

Final Notes

This guide covers everything you need to go from zero to a running AI business infrastructure.

The most important thing: **start simple, prove it works, then scale.**

One product. One audience. One system. Get that working first.

The OpenClaw Blueprint — by dossdyai Version 1.0 — 2026